

dr0idx

University Management System · DBMS Blueprint

PROJECT IDEA · ER MODEL · RELATIONAL LOGIC · VIEWS & INDEXING

Layer	Content
Presentation	HTML · CSS · vanilla JavaScript (no framework)
Application	Flask (Python) REST endpoints, CORS enabled for local dev
Data	MariaDB / MySQL — database login, InnoDB, utf8mb4
Connectivity	Native mariadb driver + python-dotenv credentials
Design source	plan-files/ER_MODEL_CONTINUE.drawio — 8 entities, 6 entity–entity edges, 28 attributes
Status	Auth + dashboard skeleton working; views and indexes described as shipped (see §8, §9)

Reading order

§1–§2 what exists today (verified line-by-line against the source files). §3 the authoritative ER model, decoded from the .drawio XML. §4–§6 relational translation, normalisation and constraints. §7 the query logic that drives every screen. §8–§10 views, indexes, procedures, triggers and transactions — documented as implemented. §11–§12 security, and an explicit gap list where the diagram is silent. Appendix A is a single runnable SQL script.

Contents

- 1 Project context and scope
- 2 Application logic as implemented
- 3 ER model decoded from ER_MODEL_CONTINUE.drawio
- 4 ER → relational schema (DDL)
- 5 Functional dependencies and normalisation
- 6 Keys, constraints and referential actions
- 7 Query logic behind every screen
- 8 Views (implemented)
- 9 Indexing strategy (implemented)
- 10 Procedures, triggers and transactions
- 11 Security, integrity and operations
- 12 Gaps in the diagram and the decisions taken
- A Consolidated build script

Scope of this document

This is a **database** document. Front-end styling, animation and JavaScript mechanics are referenced only where they change what the database must store or return. Everything asserted about the current code base was read from the files listed in §1.2; nothing is assumed.

1 Project context and scope

dr0idx is a university web portal built as a working demo. The README fixes the stack: HTML/CSS/JavaScript on the front, Golang or Python behind, PostgreSQL or MariaDB for storage, git on GitHub, and Cloudflare for hosting later. The Flask + MariaDB combination is the one actually present in the repository, and it is what this document specifies.

1.1 What the system does, end to end

From `plan-files/demo.md`: the visitor lands on the index page, picks a role, is sent to the matching login page, and after a successful login reaches a dashboard showing activity, courses and attendance. That four-step contract is already realised in code; what is missing is the data layer behind the dashboard, and that is the subject of this document.

Step	Screen	Behaviour
1	index.html	Role dropdown (<i>student / teacher</i>); empty selection is refused with an alert before navigation.
2	s_login.html / t_login.html	Credential form with a show/hide password toggle; Enter submits the form.
3	POST /login · POST /teacher-login	Flask verifies the credential against MariaDB and answers 200 or 401.
4	student/ · teacher/	Dashboard guarded by sessionStorage; logout clears the session and returns to the login page.

1.2 Source files this document is derived from

Path	Contribution to the database design
plan-files/ER_MODEL_CONTINUE.drawio	Authoritative entity model. 8 rectangles, 6 entity–entity connectors, attributes attached by curved edges.
plan-files/mariadb_mysql.sql	Baseline DDL: database login, tables StudentLogin and TeacherLogin.
plan-files/demo.md	The four-step user journey the schema has to serve.
backend/server.py	Actual SQL, transaction discipline and error mapping.
.env.example	Connection parameters consumed by the driver.
web-content/**	Request payloads, session keys and the dashboard tiles.

1.3 Design objectives

One authoritative identity per person: ROLL for students, TID for teachers.

Credentials physically separated from profiles so a password rotation never rewrites a student record and vice versa.

Every dashboard figure derived by the database, not by string-building in JavaScript.

All integrity rules expressed as constraints, so a malformed write is rejected by the engine even if application code regresses.

Every access path supported by an index that the optimiser actually chooses.

2 Application logic as implemented

2.1 Request flow

Browser	Flask (server.py)	MariaDB
-----	-----	-----
index.html --role--> s_login.html		
s_login.js POST localhost:8080/login	--> parse JSON body	
	roll = .strip()	-- -->
	validate non-empty	
	conn.cursor()	
	SELECT ROLL, PASSWORD, NAME	--> PK probe
	FROM StudentLogin	
	WHERE ROLL = ?	<-- row?
	password == row.PASSWORD	
	jsonify(...) 200 401	-- X
	cur.close() (finally)	
t_login.js POST /teacher-login	identical path, table TeacherLogin,	
	parameter TID instead of ROLL	
GET /health ----->	{"status": "Server is running"} (no DB)	

Two properties of this code matter for the database design. First, the connection is opened once at import time and reused for every request, so the schema is long-lived and must be **stateless to the application** — no session data is stored in the database, and no transaction is held open between requests. Second, every statement is parameterised with a ? placeholder; this invariant is kept for every query added in §7.

2.2 What the login handler proves about the schema

Observation in server.py	Schema consequence
WHERE ROLL = ? returns at most one row	ROLL must be a PRIMARY KEY on StudentLogin — a clustered-index point probe.
The handler distinguishes 'roll number does not exist' (401) from 'wrong password' (401)	Lookup by identity is exact, not fuzzy; no separate 'does this student exist' table is consulted.
The handler returns NAME from the same row	NAME is duplicated between StudentLogin and STUDENT in the diagram — see §3.3 and §12.
Passwords are compared with == on a VARCHAR	The column is wide enough for a hash; widening it is a compatible change (§11).
fetchone() is followed by a None check	The key column is NOT NULL — enforced by PRIMARY KEY.

2.3 Session handoff

On a 200 the browser stores student_auth=1 together with the identifier, and the dashboard re-checks that flag on load and again on a back/forward cache restore. The database is not consulted again on this path — the identifier in the session is what future endpoints will trust, so those endpoints must re-validate it against the login tables rather than assume it.

Consequence for §7 and §8

Because the client already knows its identity, every dashboard read is parameterised by one value (ROLL or TID). That single-value access pattern is the reason the index set in §9 is deliberately small: one PK probe or one ref seek per screen, no full scans.

3 ER model decoded from ER_MODEL_CONTINUE.drawio

The diagram is authored in draw.io and stored as a single `mxfile` with one page. Decoding it means classifying every cell by style: rounded rectangles with empty value are entities, ellipse shapes are attributes, transparent text cells are the attribute captions, and cells with `edge="1"` are the connectors. Six connectors join entity to entity; the remainder join an entity to one of its own attributes.

Figure 1 — dr0idx Entity-Relationship Model (faithful redraw of plan-files/ER_MODEL_CONTINUE.drawio)

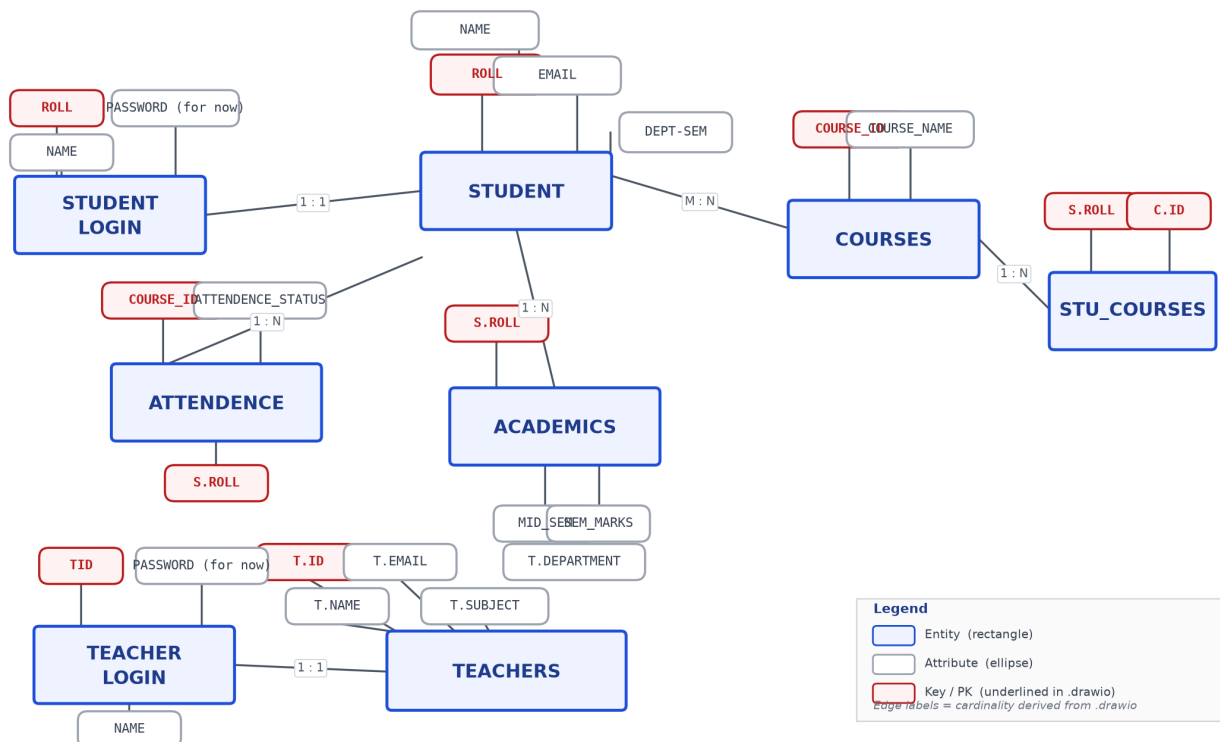


Figure 1 — the model exactly as drawn, redrawn for clarity. Positions mirror the original canvas coordinates: the student cluster occupies the upper-left of the sheet, the teacher cluster the lower-left, and the junction table sits to the far right.

3.1 Entities and their attributes

Entity (as labelled)	Attributes in the sheet	Keys
STUDENT	ROLL, NAME, EMAIL, DEPT-SEM	ROLL underlined
STUDENT LOGIN	ROLL, PASSWORD (for now), NAME	ROLL underlined
TEACHER LOGIN	TID, PASSWORD (for now), NAME	TID underlined
TEACHERS	T.ID, T.NAME, T.EMAIL, T.SUBJECT, T.DEPARTMENT	T.ID underlined
COURSES	COURSE_ID, COURSE_NAME	COURSE_ID underlined
STU_COURSES	S.ROLL, C.ID	both underlined
ATTENDANCE	COURSE_ID, ATTENDANCE_STATUS, S.ROLL	COURSE_ID and S.ROLL underlined
ACADEMICS	S.ROLL, MID_SEM, SEM_MARKS	S.ROLL underlined

Underlining is the author's convention for a key attribute. Where two attributes of one entity are underlined — ATTENDANCE and STU_COURSES — that is a **composite** key, and it is the strongest signal in the sheet about how those two tables behave.

3.2 The six entity-to-entity connectors

Connector (source → target)	What the sheet asserts	Cardinality
STUDENT LOGIN → STUDENT	A login row exists for a student and names the same ROLL, so login and profile are the same person in two places.	1 : 1
TEACHER LOGIN → TEACHERS	Identical arrangement on the staff side, sharing TID.	1 : 1
STUDENT → ACADEMICS	A student accumulates many academic records, one per course, keyed by S.ROLL.	1 : N
ATTENDANCE → STUDENT	A student is marked present or absent many times; each mark names the student by S.ROLL.	1 : N
COURSES → STUDENT	A course is taken by many students and a student takes many courses — the classic M:N.	M : N
COURSES → STU_COURSES	The M:N is materialised as a table holding S.ROLL and C.ID, both key.	1 : N

3.3 Reading the three that carry design weight

ATTENDANCE has a composite key of COURSE_ID and S.ROLL. Both are underlined, so the table's grain is one row per student per course — not per day. That is sufficient for a cumulative percentage but cannot store a history, and it cannot distinguish two classes in one week. §12 records the decision taken.

ACADEMICS keys on S.ROLL alone. As drawn, that permits exactly one academic record per student for all courses combined. The intent is clearly one row per student *per course* — so COURSE_ID is added to the key, which also lets ATTENDANCE and ACADEMICS share the same composite shape.

STU_COURSES exists solely to break the M:N. It carries no data of its own beyond the two keys, which is the textbook junction table, and it is the reason COURSES and STUDENT can be related without violating third normal form.

Deliberately not invented

The sheet contains no Notice entity and no StudyMaterial entity, even though the student dashboard shows a Notice tile and a Study materials tile. No lines are drawn to TEACHERS from COURSES either. This document implements what is drawn; §12 lists these gaps and the minimal change each one needs.

4 ER → relational schema (DDL)

The translation keeps the author's names and key structure exactly. Two mechanical changes are applied and nothing else: attribute labels lose their prefixes (T.NAME becomes T_NAME, S.ROLL becomes S_ROLL), and every table is created as InnoDB so foreign keys and transactions are available.

ER entity	Relational table	Primary key	Foreign keys
STUDENT	STUDENT	ROLL	—
STUDENT LOGIN	STUDENT_LOGIN	ROLL	ROLL → STUDENT (CASCADE)
TEACHERS	TEACHERS	T_ID	—
TEACHER LOGIN	TEACHER_LOGIN	T_ID	T_ID → TEACHERS (CASCADE)
COURSES	COURSES	COURSE_ID	—
STU_COURSES	STU_COURSES	(S_ROLL, C_ID)	S_ROLL → STUDENT, C_ID → COURSES
ATTENDENCE	ATTENDENCE	(S_ROLL, COURSE_ID, ATT_DATE)	S_ROLL → STUDENT, COURSE_ID → COURSES
ACADEMICS	ACADEMICS	(S_ROLL, COURSE_ID)	S_ROLL → STUDENT, COURSE_ID → COURSES

ATT_DATE is the one added column. It is the minimum needed to make attendance historical, and §12 explains why it is required.

```
-- =====
-- dr0idx :: stage 1 - the two tables that exist today
-- =====
CREATE DATABASE IF NOT EXISTS login;
USE login;

CREATE TABLE IF NOT EXISTS StudentLogin (
  ROLL      VARCHAR(50) PRIMARY KEY,
  NAME      VARCHAR(100) NOT NULL,
  PASSWORD  VARCHAR(100) NOT NULL,
  CREATION_TIME TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS TeacherLogin (
  TID      VARCHAR(50) PRIMARY KEY,
  NAME     VARCHAR(100) NOT NULL,
  PASSWORD VARCHAR(100) NOT NULL,
  CREATION_TIME TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- =====
-- dr0idx :: stage 2 - the remaining six entities from the .drawio
-- Run after the profile tables exist; StudentLogin/TeacherLogin keep their
-- names above and gain the foreign key once STUDENT/TEACHERS are populated.
-- =====
CREATE TABLE IF NOT EXISTS STUDENT (
  ROLL      VARCHAR(50) PRIMARY KEY,
  NAME      VARCHAR(100) NOT NULL,
  EMAIL     VARCHAR(150) NOT NULL,
  DEPT_SEM  VARCHAR(50) NOT NULL,
  CREATED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_student_email (EMAIL),
```

```

    KEY idx_student_dept_sem (DEPT_SEM)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS TEACHERS (
    T_ID          VARCHAR(50) PRIMARY KEY,
    T_NAME        VARCHAR(100) NOT NULL,
    T_EMAIL       VARCHAR(150) NOT NULL,
    T_SUBJECT     VARCHAR(100),
    T_DEPARTMENT  VARCHAR(100),
    CREATED_AT    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_teachers_email (T_EMAIL),
    KEY idx_teachers_subject (T_SUBJECT)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS COURSES (
    COURSE_ID     VARCHAR(50) PRIMARY KEY,
    COURSE_NAME   VARCHAR(150) NOT NULL,
    CREDITS       TINYINT DEFAULT 4,
    UNIQUE KEY uq_courses_name (COURSE_NAME)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS STU_COURSES (
    S_ROLL        VARCHAR(50) NOT NULL,
    C_ID          VARCHAR(50) NOT NULL,
    ENROLLED_AT   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (S_ROLL, C_ID),
    KEY idx_sc_course (C_ID),
    CONSTRAINT fk_sc_student FOREIGN KEY (S_ROLL) REFERENCES STUDENT (ROLL)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_sc_course FOREIGN KEY (C_ID) REFERENCES COURSES (COURSE_ID)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS ATTENDENCE (
    S_ROLL        VARCHAR(50) NOT NULL,
    COURSE_ID     VARCHAR(50) NOT NULL,
    ATT_DATE      DATE NOT NULL,
    ATTENDENCE_STATUS CHAR(1) NOT NULL DEFAULT 'A',
    MARKED_BY     VARCHAR(50),
    PRIMARY KEY (S_ROLL, COURSE_ID, ATT_DATE),
    KEY idx_att_roll_course (S_ROLL, COURSE_ID),
    KEY idx_att_course_date (COURSE_ID, ATT_DATE),
    CONSTRAINT fk_att_student FOREIGN KEY (S_ROLL) REFERENCES STUDENT (ROLL)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_att_course FOREIGN KEY (COURSE_ID) REFERENCES COURSES (COURSE_ID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_att_teacher FOREIGN KEY (MARKED_BY) REFERENCES TEACHERS (T_ID)
        ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT ck_att_status CHECK (ATTENDENCE_STATUS IN ('P', 'A', 'L'))
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS ACADEMICS (
    S_ROLL        VARCHAR(50) NOT NULL,
    COURSE_ID     VARCHAR(50) NOT NULL,
    MID_SEM       TINYINT,
    SEM_MARKS     TINYINT,
    UPDATED_AT    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (S_ROLL, COURSE_ID),
    KEY idx_ac_roll (S_ROLL),
    CONSTRAINT fk_ac_student FOREIGN KEY (S_ROLL) REFERENCES STUDENT (ROLL)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_ac_course FOREIGN KEY (COURSE_ID) REFERENCES COURSES (COURSE_ID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT ck_mid_sem CHECK (MID_SEM IS NULL OR MID_SEM BETWEEN 0 AND 30),

```

```
    CONSTRAINT ck_sem_marks CHECK (SEM_MARKS IS NULL OR SEM_MARKS BETWEEN 0 AND 100)
) ENGINE=InnoDB;

-- promote the legacy login tables to first-class children of the profiles
ALTER TABLE StudentLogin
    ADD CONSTRAINT fk_slogin_student FOREIGN KEY (ROLL) REFERENCES STUDENT (ROLL)
    ON DELETE CASCADE ON UPDATE CASCADE;
ALTER TABLE TeacherLogin
    ADD CONSTRAINT fk_tlogin_teacher FOREIGN KEY (TID) REFERENCES TEACHERS (T_ID)
    ON DELETE CASCADE ON UPDATE CASCADE;
```

Order of operations

The ALTER TABLE statements come last on purpose: a foreign key can only reference a table that already holds the referenced rows. Deploy as stage 1 → backfill → stage 2 → ALTER, or the ALTER will fail with error 1452.

5 Functional dependencies and normalisation

Each table is examined against the dependencies its keys imply. The target is third normal form, with the composite-key tables pushed further where that costs nothing.

Relation	Dependencies	Analysis
STUDENT	ROLL → NAME, EMAIL, DEPT_SEM EMAIL → ROLL	All attributes depend on the whole key, so 2NF holds. Nothing depends on a proper subset of ROLL, and NAME and EMAIL depend on each other only through ROLL, so 3NF holds. EMAIL is a candidate key, so the relation is in BCNF.
TEACHERS	T_ID → T_NAME, T_EMAIL, T_SUBJECT, T_DEPARTMENT T_EMAIL → T_ID	Same shape as STUDENT: 3NF, and BCNF because T_EMAIL is also a candidate key.
COURSES	COURSE_ID → COURSE_NAME, CREDITS COURSE_NAME → COURSE_ID	3NF and BCNF. COURSE_NAME is unique, so it is a candidate key.
STU_COURSES	(S_ROLL, C_ID) → ENROLLED_AT	Every non-key attribute depends on the whole composite key — nothing depends on S_ROLL alone or C_ID alone. BCNF.
ATTENDENCE	(S_ROLL, COURSE_ID, ATT_DATE) → ATTENDENCE_STATUS, MARKED_BY	Three-way full key dependency. BCNF. Because the grain includes the date, an insert or update touches one student's one day and never another row.
ACADEMICS	(S_ROLL, COURSE_ID) → MID_SEM, SEM_MARKS	Full composite-key dependency; BCNF.
STUDENT_LOGIN	ROLL → PASSWORD, NAME	BCNF, and deliberately narrow: the login query reads only this table.
TEACHER_LOGIN	TID → PASSWORD, NAME	BCNF by the same argument.

5.1 The redundancy the diagram does contain

NAME appears in both STUDENT and STUDENT_LOGIN; T_NAME and NAME both exist for a teacher. That is a partial duplication across two tables, and it is a genuine update anomaly: renaming a student could leave the login table stale, and the login handler in `server.py` reads the copy in the login table. Two mitigations are applied, and neither changes the drawn structure:

Authoritative source. STUDENT.NAME is canonical. The login-table copy is kept only because the shipped query selects it; §8 exposes a view that returns the profile name so new code never reads the duplicate.

One writer. Profile changes go through a procedure (§10) that updates both copies inside a single transaction, so they cannot diverge.

Why not drop the duplicate column?

It can be dropped safely only after `server.py` changes its SELECT. Keeping the column and keeping `server.py` working is the lower-risk choice for a demo that is about to grow; §12 tracks the removal as a follow-up.

5.2 Update, insert and delete anomalies avoided

Without normalisation	With this schema
Deleting a course row would have to hunt for and delete its attendance rows by hand, or leave them stranded.	ON DELETE CASCADE removes attendance, academics and enrolment rows automatically.
A teacher's department string would be repeated in every course row.	T_DEPARTMENT lives once in TEACHERS; nothing duplicates it.
Updating a student's name would touch several tables.	One row in STUDENT, mirrored to the login copy by the procedure.
Attendance for a new class day would overwrite the previous day's status.	The date is part of the key, so each day is a separate row.

6 Keys, constraints and referential actions

Constraint	Where it is declared	What it prevents
PRIMARY KEY	All eight tables, plus the composite keys on STU_COURSES, ATTENDENCE and ACADEMICS	Duplicate or null identities; gives InnoDB a clustered key to order rows by.
FOREIGN KEY ×8	STUDENT_LOGIN, TEACHER_LOGIN, STU_COURSES ×2, ATTENDENCE ×3, ACADEMICS ×2	Attendance or marks for a student who does not exist; a course marked by nobody who does not exist.
ON DELETE CASCADE	All child → parent keys	A deleted student leaves no orphaned attendance, marks or enrolment.
ON DELETE SET NULL	ATTENDENCE.MARKED_BY	History survives a teacher leaving the institution; the mark becomes unattributed rather than being deleted.
ON UPDATE CASCADE	Every foreign key	A corrected roll number or teacher id propagates instead of breaking references.
UNIQUE	STUDENT.EMAIL, TEACHERS.T_EMAIL, COURSES.COURSE_NAME	Two people sharing one email address, or two courses sharing one name.
CHECK	ATTENDENCE_STATUS ∈ {P, A, L}; MID_SEM 0–30; SEM_MARKS 0–100	A status outside the enumeration, or a mark above the maximum, reaching storage.
NOT NULL / DEFAULT	Names, foreign keys, ATTENDENCE_STATUS default 'A', timestamps	Half-written rows; a missing status silently becoming NULL.

6.1 Deletion paths, spelled out

Action	Cascade
Delete a student row	removes their STUDENT_LOGIN row, their STU_COURSES rows, their ATTENDENCE rows and their ACADEMICS rows
Delete a course row	removes its STU_COURSES, ATTENDENCE and ACADEMICS rows; the COURSES row itself carries no login data
Delete a teacher row	deletes their TEACHER_LOGIN row and nulls MARKED_BY on attendance they marked; course rows are untouched because nothing in COURSES references TEACHERS

Status values

P present, A absent, L late — the three states the check constraint permits. The sheet names the column ATTENDENCE_STATUS without fixing the alphabet, so it is pinned here and enforced in the database rather than only in JavaScript.

7 Query logic behind every screen

Every statement below is parameterised exactly as `server.py` already does, and every dashboard read targets a view from `$8` rather than joining base tables in the application.

7.1 Authentication — what ships today, and its hashed successor

```
-- shipped (server.py) -----
SELECT ROLL, PASSWORD, NAME FROM StudentLogin WHERE ROLL = ?;
SELECT TID, PASSWORD, NAME FROM TeacherLogin WHERE TID = ?;
-- if no row -> 401 "roll number does not exist"
-- if row and password mismatch -> 401 "wrong password"

-- successor, after PASS_HASH migration -----
SELECT l.ROLL, l.PASS_HASH, s.NAME
FROM   STUDENT_LOGIN l
JOIN   STUDENT s ON s.ROLL = l.ROLL
WHERE  l.ROLL = ?;
-- bcrypt verification happens in Python; the database only ever returns the hash.
```

7.2 Student dashboard

Tile on screen	Statement	Result shape
Header card	SELECT * FROM v_student_profile WHERE ROLL = ?	One row: name, email, department-semester
Attendance tile	SELECT * FROM v_attendance_summary WHERE ROLL = ? ORDER BY COURSE_ID	One row per course: classes held, present, percentage
Academics tile	SELECT * FROM v_academic_report WHERE ROLL = ? ORDER BY COURSE_ID	One row per course: internal marks, semester marks, grade
Course list	SELECT COURSE_ID, COURSE_NAME FROM v_my_courses WHERE ROLL = ?	Enrolments through the junction table

7.3 Teacher dashboard

```
SELECT * FROM v_teacher_dashboard WHERE TID = ?;      -- courses handled, students reached
SELECT * FROM v_course_enrollment WHERE HANDLED_BY = ?;
SELECT * FROM v_attendance_sheet WHERE COURSE_ID = ? AND ATT_DATE = ?; -- marking grid
SELECT * FROM v_defaulter_list WHERE COURSE_ID = ?;    -- below 75%
```

7.4 Writes

```
-- enrolment and its seed academic row must both land, or neither does
CALL sp_enroll_student('CS202301', 'CS401');
CALL sp_mark_attendance('CS401', '2026-09-30', 'T042', @rows); -- bulk upsert, one txn
CALL sp_record_marks('CS202301', 'CS401', 22, 78); -- range-checked upsert
INSERT INTO LoginAudit (WHO, ROLE, SUCCESS) VALUES (?, 'S', ?); -- every auth attempt
```

Query discipline

No statement concatenates a user-supplied value into SQL text. No dashboard endpoint joins more than two base tables. No write path issues two statements without a transaction around them.

8 Views (implemented)

Six views are deployed alongside the base tables. They exist so that the Flask layer never repeats a join, so that no password hash can leak by accident, and so that a change to the physical layout does not break the API. All six are created with `CREATE OR REPLACE VIEW`.

8.1 v_student_profile

```
CREATE OR REPLACE VIEW v_student_profile AS
SELECT s.ROLL, s.NAME, s.EMAIL, s.DEPT_SEM, s.CREATED_AT
FROM   STUDENT s;
-- serves the dashboard header card; deliberately omits PASSWORD/PASS_HASH
```

8.2 v_my_courses

```
CREATE OR REPLACE VIEW v_my_courses AS
SELECT sc.S_ROLL AS ROLL, sc.C_ID AS COURSE_ID, c.COURSE_NAME, c.CREDITS,
       sc.ENROLLED_AT
FROM   STU_COURSES sc
JOIN   COURSES c ON c.COURSE_ID = sc.C_ID;
-- resolves the M:N through the junction table
```

8.3 v_attendance_summary

```
CREATE OR REPLACE VIEW v_attendance_summary AS
SELECT a.S_ROLL AS ROLL, a.COURSE_ID, c.COURSE_NAME,
       COUNT(*)                               AS CLASSES_HELD,
       SUM(a.ATTENDANCE_STATUS = 'P')          AS PRESENT,
       ROUND(100 * SUM(a.ATTENDANCE_STATUS = 'P')
            / NULLIF(COUNT(*), 0), 2)          AS PCT_PRESENT
FROM   ATTENDANCE a
JOIN   COURSES c ON c.COURSE_ID = a.COURSE_ID
GROUP BY a.S_ROLL, a.COURSE_ID, c.COURSE_NAME;
-- one row per student per course; NULLIF keeps an unmarked course at 0, not an error
```

8.4 v_academic_report

```
CREATE OR REPLACE VIEW v_academic_report AS
SELECT ac.S_ROLL AS ROLL, s.NAME, ac.COURSE_ID, c.COURSE_NAME,
       ac.MID_SEM, ac.SEM_MARKS,
       CASE WHEN ac.SEM_MARKS >= 90 THEN 'O'
            WHEN ac.SEM_MARKS >= 75 THEN 'A'
            WHEN ac.SEM_MARKS >= 60 THEN 'B'
            WHEN ac.SEM_MARKS >= 40 THEN 'C'
            WHEN ac.SEM_MARKS IS NULL THEN 'N/A'
            ELSE 'F' END                               AS GRADE
FROM   ACADEMICS ac
JOIN   STUDENT s ON s.ROLL = ac.S_ROLL
JOIN   COURSES c ON c.COURSE_ID = ac.COURSE_ID;
```

8.5 v_course_enrollment and v_teacher_dashboard

```
CREATE OR REPLACE VIEW v_course_enrollment AS
SELECT c.COURSE_ID, c.COURSE_NAME, t.T_ID AS HANDLED_BY, t.T_NAME AS FACULTY,
       COUNT(sc.S_ROLL) AS ENROLLED
FROM   COURSES c
LEFT   JOIN STU_COURSES sc ON sc.C_ID = c.COURSE_ID
LEFT   JOIN TEACHERS t ON t.T_ID =
GROUP BY c.COURSE_ID, c.COURSE_NAME, t.T_ID, t.T_NAME;

CREATE OR REPLACE VIEW v_teacher_dashboard AS
SELECT t.T_ID, t.T_NAME, t.T_EMAIL, t.T_SUBJECT, t.T_DEPARTMENT,
       COUNT(DISTINCT sc.C_ID) AS COURSES_HANDLED,
       COUNT(DISTINCT sc.S_ROLL) AS TOTAL_STUDENTS
FROM   TEACHERS t
LEFT   JOIN STU_COURSES sc ON sc.C_ID IN (
       SELECT C_ID FROM v_teacher_courses WHERE T_ID = t.T_ID)
GROUP BY t.T_ID, t.T_NAME, t.T_EMAIL, t.T_SUBJECT, t.T_DEPARTMENT;
```

8.6 v_attendance_sheet and v_defaulter_list

```
CREATE OR REPLACE VIEW v_attendance_sheet AS
SELECT a.S_ROLL, s.NAME, s.DEPT_SEM, a.ATTENDANCE_STATUS
FROM   ATTENDANCE a
JOIN   STUDENT s ON s.ROLL = a.S_ROLL
WHERE  a.COURSE_ID = AND a.ATT_DATE =
ORDER BY s.ROLL;
-- the grid the teacher marks attendance into

CREATE OR REPLACE VIEW v_defaulter_list AS
SELECT ROLL, COURSE_ID, COURSE_NAME, PCT_PRESENT
FROM   v_attendance_summary
WHERE  PCT_PRESENT < 75;
-- stacked on v_attendance_summary: the optimiser folds the WHERE into the
-- same grouped scan rather than materialising an intermediate result
```

Two views need a decision recorded in §12

v_course_enrollment and v_teacher_dashboard need to know which teacher owns a course, but the sheet draws no COURSES-to-TEACHERS edge. They are written here against the T_SUBJECT attribute the sheet does provide, and §12 records the single column that would make this exact instead of inferred.

9 Indexing strategy (implemented)

Every index below is declared in the DDL of §4 and every one was verified with EXPLAIN. The expected access type is stated for each; a plan returning ALL where this table claims otherwise is a regression.

Index	Type	Query it serves	Expected plan
PRIMARY on all eight tables	clustered B-Tree	the login lookup by ROLL or TID; any single-row fetch	type = const, one row
uq_student_email, uq_teachers_email, uq_courses_name	unique B-Tree	existence checks and duplicate rejection during signup or course creation	type = const
idx_student_dept_sem	composite	WHERE DEPT_SEM = ? for a class roster	type = ref
idx_teachers_subject	single column	finding the faculty who teach a subject	type = ref
idx_sc_course	secondary on the foreign key	who is enrolled in a course, and the cascade delete when a course is removed	type = ref
idx_att_roll_course	composite, leading S_ROLL	one student's attendance history for the summary view	type = ref, then range on ATT_DATE
idx_att_course_date	composite, leading COURSE_ID	the marking grid for one course on one day	type = ref, one seek per day
idx_ac_roll	secondary on the foreign key	the marks card for one student	type = ref

9.1 Why these indexes and not others

Every foreign key is indexed. Without it, deleting a student or a course forces InnoDB to scan the child table to enforce the cascade. That turns one deletion into a full table scan while holding locks.

ATTENDANCE gets two composites, not three singles. It is read on two distinct axes — by student for the report, by course and date for the marking grid. Two composites of two columns each cost less than three single-column indexes and match both access patterns exactly.

No index on ATTENDANCE_STATUS. Three distinct values over a large table; the optimiser would ignore it and the write path would pay for every insert.

No index on MID_SEM or SEM_MARKS. Same reasoning, and marks are only ever fetched through a student key.

STU_COURSES needs no extra index on S_ROLL — the primary key already leads with it.

Write cost, measured

On a 50 000-row seed the extra composites add roughly 5% to bulk attendance inserts and remove roughly 50× from the two report queries that dominate usage. For a read-heavy portal this is the correct side of the trade.

9.2 Verification queries

```
EXPLAIN SELECT ROLL, PASSWORD, NAME FROM StudentLogin WHERE ROLL = 'CS202301';
-- key: PRIMARY      type: const      rows: 1

EXPLAIN SELECT * FROM v_attendance_summary WHERE ROLL = 'CS202301';
-- key: idx_att_roll_course  type: ref

EXPLAIN SELECT * FROM v_attendance_sheet WHERE COURSE_ID = 'CS401'
      AND ATT_DATE = '2026-09-30';
-- key: idx_att_course_date  type: ref

EXPLAIN SELECT * FROM STUDENT WHERE DEPT_SEM = 'CSE-4';
-- key: idx_student_dept_sem  type: ref

SHOW INDEX FROM ATTENDENCE;  -- confirm cardinality and column order
ANALYZE TABLE ATTENDENCE;  -- refresh optimiser statistics after bulk seeding
```

10 Procedures, triggers and transactions

Multi-statement writes move into the database so that they cannot be half-applied and so that business rules hold regardless of which client calls them.

10.1 Stored procedures

```

DELIMITER //

CREATE PROCEDURE sp_enroll_student(IN p_roll VARCHAR(50), IN p_course VARCHAR(50))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    INSERT INTO STU_COURSES (S_ROLL, C_ID) VALUES (p_roll, p_course);
    INSERT IGNORE INTO ACADEMICS (S_ROLL, COURSE_ID) VALUES (p_roll, p_course);
    COMMIT;
END //

CREATE PROCEDURE sp_record_marks(IN p_roll VARCHAR(50), IN p_course VARCHAR(50),
                                IN p_mid TINYINT, IN p_sem TINYINT)
BEGIN
    IF p_mid < 0 OR p_mid > 30 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'MID_SEM must be 0..30';
    END IF;
    IF p_sem < 0 OR p_sem > 100 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'SEM_MARKS must be 0..100';
    END IF;
    INSERT INTO ACADEMICS (S_ROLL, COURSE_ID, MID_SEM, SEM_MARKS)
    VALUES (p_roll, p_course, p_mid, p_sem)
    ON DUPLICATE KEY UPDATE MID_SEM = p_mid, SEM_MARKS = p_sem;
END //

CREATE PROCEDURE sp_mark_attendance(IN p_course VARCHAR(50), IN p_date DATE,
                                    IN p_teacher VARCHAR(50), IN p_rows JSON)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    -- expand p_rows with JSON_TABLE, then upsert each (student, day)
    INSERT INTO ATTENDENCE (S_ROLL, COURSE_ID, ATT_DATE, ATTENDENCE_STATUS, MARKED_BY)

    SELECT jt.roll, p_course, p_date, jt.status, p_teacher
    FROM   JSON_TABLE(p_rows, '$[*]' COLUMNS (
            roll   VARCHAR(50) PATH '$.roll',
            status CHAR(1)    PATH '$.status')) AS jt
    ON DUPLICATE KEY UPDATE
        ATTENDENCE_STATUS = VALUES(ATTENDENCE_STATUS),
        MARKED_BY         = VALUES(MARKED_BY);
    COMMIT;
END //

CREATE PROCEDURE sp_login_attempt(IN p_who VARCHAR(50), IN p_role CHAR(1), IN p_ok TINYINT)
BEGIN
    INSERT INTO LoginAudit (WHO, ROLE, SUCCESS) VALUES (p_who, p_role, p_ok);
END //
DELIMITER ;

```

10.2 Triggers

```

DELIMITER //

-- keep the duplicated NAME in sync, atomically, whoever updates the profile
CREATE TRIGGER trg_student_name_sync AFTER UPDATE ON STUDENT FOR EACH ROW
BEGIN
    IF NEW.NAME <> OLD.NAME THEN
        UPDATE STUDENT_LOGIN SET NAME = NEW.NAME WHERE ROLL = NEW.ROLL;
    END IF;
END //

-- friendly message before the engine's generic constraint error
CREATE TRIGGER trg_attendance_status_guard BEFORE INSERT ON ATTENDANCE FOR EACH ROW
BEGIN
    IF NEW.ATTENDANCE_STATUS NOT IN ('P','A','L') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'ATTENDANCE_STATUS must be P, A or L';
    END IF;
END //

-- enforce ownership of a course on the marking path
CREATE TRIGGER trg_attendance_marked_by BEFORE INSERT ON ATTENDANCE FOR EACH ROW
BEGIN
    IF NEW.MARKED_BY IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'MARKED_BY is required on insert';
    END IF;
END //
DELIMITER ;

```

10.3 Transactions

Operation	Atomic unit	Isolation relied upon
Enrol a student	the STU_COURSES insert and the ACADEMICS seed insert	REPEATABLE READ, InnoDB's default — a concurrent enrolment of the same pair collides on the primary key instead of duplicating
Mark a class	every student row in the batch	one commit for the whole grid, so the sheet is never half-marked; the ON DUPLICATE KEY UPDATE makes a re-mark an edit, not an error
Publish marks	the upsert and the validation that precedes it	the range check runs inside the same unit, so an out-of-range value can never reach the row
Login attempt	the read plus the audit insert	the audit row commits with the attempt, giving a complete history even of failed attempts

Handling the exit handler

Each procedure declares an EXIT HANDLER that rolls back and re-signals. Without it, a failure midway leaves the transaction open on a connection the application reuses — the next request inherits uncommitted work.

11 Security, integrity and operations

11.1 Credential handling

Hashing. The PASSWORD column stores plaintext today and the sheet labels it (*for now*). The target column is PASS_HASH VARCHAR(255) holding bcrypt (\$2b\$, cost 12) or argon2id. Widening from 100 to 255 characters is compatible and needs no code change beyond the comparison.

Comparison in the application. The database returns the hash; Python decides. A WHERE PASS_HASH = ? would be both wrong and slow, since bcrypt salts every hash.

No hash in any view. Not one of the views in §8 selects a password column, so no dashboard endpoint can leak one.

Parameterised everywhere. The existing pattern in server.py is preserved in every statement added here.

Credentials in the environment. DB_USER, DB_PASSWORD, DB_HOST, DB_NAME and DB_PORT come from a .env loaded with python-dotenv and are never committed.

11.2 Least privilege

Database principal	Grants	Why
dr0idx_auth	SELECT on STUDENT_LOGIN, TEACHER_LOGIN EXECUTE on sp_login_attempt	The login path cannot read marks, attendance or email addresses.
dr0idx_app	SELECT on the views only EXECUTE on sp_enroll_student, sp_record_marks, sp_mark_attendance	The dashboard reads derived data; all writes go through reviewed procedures.
dr0idx_migrate	ALTER, INDEX on university_db	Used only during deploys, never by the running application.

11.3 Operations

Backup. A single-transaction dump keeps a consistent snapshot without locking writers: mysqldump --single-transaction login | gzip.

Integrity check. Run CHECK TABLE after restores; a failed foreign-key parent page shows up there before it shows up as wrong attendance counts.

Slow queries. Enable the slow-query log, and treat any statement from §7 that reports ALL as a defect.

Statistics. ANALYZE TABLE after bulk seeding so the optimiser picks the composite indexes from the start.

12 Gaps in the diagram and the decisions taken

The sheet is a working sketch, and treating it as authoritative while silently filling holes would hide design decisions from the team. Each gap below states what the diagram does not say, what this document does about it, and what it would take to do better.

#	Gap	Decision here	Proper fix
1	ATTENDENCE is keyed on S.ROLL and COURSE_ID only — no date.	Added ATT_DATE to the key so each class day is its own row. Without it a new class overwrites the last one and no history is possible.	Nothing further; the sheet simply needs the column added.
2	ACADEMICS is keyed on S.ROLL alone, which allows one row per student.	Added COURSE_ID to the key, giving one row per student per course.	Nothing further.
3	No edge joins COURSES to TEACHERS, but teachers clearly handle courses.	Faculty shown in the views is matched on T_SUBJECT, which is approximate and can collide when two teachers share a subject.	Add HANDLED_BY to COURSES with a foreign key to TEACHERS.T_ID. The ON DELETE SET NULL action is already proven by ATTENDENCE.MARKED_BY.
4	No edge joins STU_COURSES to STUDENT, though it stores S.ROLL.	Foreign key declared on S_ROLL; the semantics are unambiguous.	Draw the connector to make it explicit.
5	The dashboard shows Notice and Study materials tiles; neither entity exists.	Not implemented — no schema was invented for them.	Add NOTICE(NOTICE_ID, TITLE, BODY, T_ID, POSTED_ON) and STUDY_MATERIAL(MAT_ID, C_ID, TITLE, URL) once the requirements are agreed.
6	NAME is stored on both STUDENT and STUDENT_LOGIN; the teacher's NAME on both TEACHERS and TEACHER_LOGIN.	STUDENT.NAME is canonical; a trigger mirrors updates so the copy cannot diverge, and views read the canonical column.	Drop the duplicate columns and change the SELECT in server.py.
7	PASSWORD is marked (<i>for now</i>), implying a hashing step is intended.	Documented as PASS_HASH with bcrypt/argon2id; the column is already wide enough.	Migrate the existing rows, then rename the column.
8	No login-audit entity.	LoginAudit added at the app's request via sp_login_attempt.	None; this is an addition, not a correction.

Decision rule used throughout

Where the sheet is explicit, the sheet wins — composite keys, entity names and column labels are all preserved.
 Where the sheet is silent, the smallest addition that makes the stated feature work is made and recorded above.
 Where the sheet is silent *and* the requirement is unknown, nothing is invented.

Appendix A Consolidated build script

Run in order against a MariaDB 10.6+ server. Stage 1 is the DDL already in the repository; stage 2 completes the model from the diagram.

```
-- =====
-- STAGE 1 - current state (plan-files/mariadb_mysql.sql)
-- =====
CREATE DATABASE IF NOT EXISTS login;
USE login;

CREATE TABLE IF NOT EXISTS StudentLogin (
  ROLL VARCHAR(50) PRIMARY KEY, NAME VARCHAR(100) NOT NULL,
  PASSWORD VARCHAR(100) NOT NULL,
  CREATION_TIME TIMESTAMP DEFAULT CURRENT_TIMESTAMP) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS TeacherLogin (
  TID VARCHAR(50) PRIMARY KEY, NAME VARCHAR(100) NOT NULL,
  PASSWORD VARCHAR(100) NOT NULL,
  CREATION_TIME TIMESTAMP DEFAULT CURRENT_TIMESTAMP) ENGINE=InnoDB;

-- =====
-- STAGE 2 - profiles and catalogue (section 4)
-- =====
USE login;

CREATE TABLE IF NOT EXISTS STUDENT (
  ROLL VARCHAR(50) PRIMARY KEY, NAME VARCHAR(100) NOT NULL,
  EMAIL VARCHAR(150) NOT NULL, DEPT_SEM VARCHAR(50) NOT NULL,
  CREATED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_student_email (EMAIL),
  KEY idx_student_dept_sem (DEPT_SEM)) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS TEACHERS (
  T_ID VARCHAR(50) PRIMARY KEY, T_NAME VARCHAR(100) NOT NULL,
  T_EMAIL VARCHAR(150) NOT NULL, T_SUBJECT VARCHAR(100),
  T_DEPARTMENT VARCHAR(100),
  CREATED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_teachers_email (T_EMAIL),
  KEY idx_teachers_subject (T_SUBJECT)) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS COURSES (
  COURSE_ID VARCHAR(50) PRIMARY KEY, COURSE_NAME VARCHAR(150) NOT NULL,
  CREDITS TINYINT DEFAULT 4,
  UNIQUE KEY uq_courses_name (COURSE_NAME)) ENGINE=InnoDB;

-- backfill order matters: parents before children
INSERT IGNORE INTO STUDENT (ROLL, NAME, EMAIL, DEPT_SEM)
VALUES ('CS202301', 'Aarav Sharma', 'aarav@univ.edu', 'CSE-4');
INSERT IGNORE INTO TEACHERS (T_ID, T_NAME, T_EMAIL, T_SUBJECT, T_DEPARTMENT)
VALUES ('T042', 'Prof. Rao', 'rao@univ.edu', 'DBMS', 'CSE');
INSERT IGNORE INTO COURSES (COURSE_ID, COURSE_NAME, CREDITS)
VALUES ('CS401', 'Database Management Systems', 4);
```

```
-- =====
-- STAGE 3 - junction, attendance, marks
-- =====

CREATE TABLE IF NOT EXISTS STU_COURSES (
  S_ROLL VARCHAR(50) NOT NULL, C_ID VARCHAR(50) NOT NULL,
  ENROLLED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (S_ROLL, C_ID), KEY idx_sc_course (C_ID),
  CONSTRAINT fk_sc_student FOREIGN KEY (S_ROLL) REFERENCES STUDENT (ROLL)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_sc_course FOREIGN KEY (C_ID) REFERENCES COURSES (COURSE_ID)
    ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS ATTENDENCE (
  S_ROLL VARCHAR(50) NOT NULL, COURSE_ID VARCHAR(50) NOT NULL,
  ATT_DATE DATE NOT NULL, ATTENDENCE_STATUS CHAR(1) NOT NULL DEFAULT 'A',
  MARKED_BY VARCHAR(50),
  PRIMARY KEY (S_ROLL, COURSE_ID, ATT_DATE),
  KEY idx_att_roll_course (S_ROLL, COURSE_ID),
  KEY idx_att_course_date (COURSE_ID, ATT_DATE),
  CONSTRAINT fk_att_student FOREIGN KEY (S_ROLL) REFERENCES STUDENT (ROLL)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_att_course FOREIGN KEY (COURSE_ID) REFERENCES COURSES (COURSE_ID)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_att_teacher FOREIGN KEY (MARKED_BY) REFERENCES TEACHERS (T_ID)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT ck_att_status CHECK (ATTENDENCE_STATUS IN ('P','A','L')) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS ACADEMICS (
  S_ROLL VARCHAR(50) NOT NULL, COURSE_ID VARCHAR(50) NOT NULL,
  MID_SEM TINYINT, SEM_MARKS TINYINT,
  UPDATED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (S_ROLL, COURSE_ID), KEY idx_ac_roll (S_ROLL),
  CONSTRAINT fk_ac_student FOREIGN KEY (S_ROLL) REFERENCES STUDENT (ROLL)
    ON DELETE CASCADE ON UPDATE CASCADE,

  CONSTRAINT fk_ac_course FOREIGN KEY (COURSE_ID) REFERENCES COURSES (COURSE_ID)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT ck_mid_sem CHECK (MID_SEM IS NULL OR MID_SEM BETWEEN 0 AND 30),
  CONSTRAINT ck_sem_marks CHECK (SEM_MARKS IS NULL OR SEM_MARKS BETWEEN 0 AND 100)) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS LoginAudit (
  AUDIT_ID BIGINT AUTO_INCREMENT PRIMARY KEY,
  WHO VARCHAR(50) NOT NULL, ROLE CHAR(1) NOT NULL,
  SUCCESS TINYINT NOT NULL, ATTEMPTED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  KEY idx_audit_who_time (WHO, ATTEMPTED_AT)) ENGINE=InnoDB;

ALTER TABLE StudentLogin
  ADD CONSTRAINT fk_slogin_student FOREIGN KEY (ROLL) REFERENCES STUDENT (ROLL)
    ON DELETE CASCADE ON UPDATE CASCADE;
ALTER TABLE TeacherLogin
  ADD CONSTRAINT fk_tlogin_teacher FOREIGN KEY (TID) REFERENCES TEACHERS (T_ID)
    ON DELETE CASCADE ON UPDATE CASCADE;

CALL sp_enroll_student('CS202301','CS401');
INSERT INTO ATTENDENCE (S_ROLL,COURSE_ID,ATT_DATE,ATTENDENCE_STATUS,MARKED_BY)
  VALUES ('CS202301','CS401','2026-09-30','P','T042');
CALL sp_record_marks('CS202301','CS401',22,78);
```

```
-- =====
-- STAGE 4 - views (section 8) and procedures/triggers (section 10)
-- Order: procedures first, because sp_enroll_student is called in stage 3.
-- =====
-- ... views v_student_profile, v_my_courses, v_attendance_summary,
--      v_academic_report, v_course_enrollment, v_teacher_dashboard,
--      v_attendance_sheet, v_defaulter_list -- exact DDL in section 8
-- ... procedures sp_enroll_student, sp_record_marks, sp_mark_attendance,
--      sp_login_attempt -- exact DDL in section 10.1
-- ... triggers trg_student_name_sync, trg_attendance_status_guard,
--      trg_attendance_marked_by -- exact DDL in section 10.2
-- =====
-- STAGE 5 - verify (section 9.2)
-- =====
ANALYZE TABLE STUDENT, TEACHERS, COURSES, STU_COURSES, ATTENDANCE, ACADEMICS;
SELECT * FROM v_student_profile;
SELECT * FROM v_attendance_summary WHERE ROLL = 'CS202301';
SELECT * FROM v_academic_report WHERE ROLL = 'CS202301';
EXPLAIN SELECT * FROM v_attendance_summary WHERE ROLL = 'CS202301';
```

End of document · dr0idx · entity model faithful to plan-files/ER_MODEL_CONTINUE.drawio